

SIMATS ENGINEERING
ASSESSMENT TOOL 3
MODELLING COMPARATIVE ANALYSIS

COURSE NAME
ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS

COURSE CODE: MLA01

SELECTED PROBLEM
AUTOMOBILE FAULT DIAGNOSIS

Student Name	S. Roshni
Register Number	192525185
Course Outcome	CO5
Assessment Weightage	40%

Student Signature: S. Roshni

Academic Year: 2026

Code of Conduct

I, S. Roshni (192525185), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

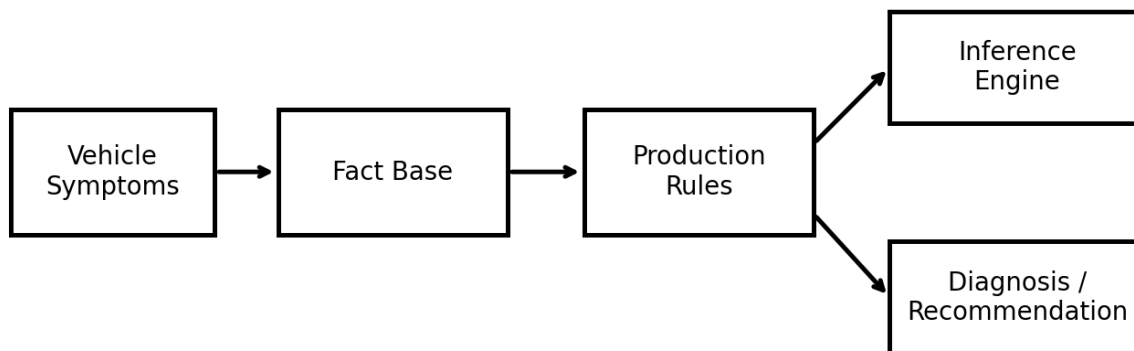
Signature of the student: S. Roshni

1. Problem Statement & Objectives

An automobile service center needs a knowledge-based diagnostic system that can identify likely vehicle faults from observable symptoms such as engine overheating, starting failure, abnormal noise, low mileage and dashboard warning indicators. The system should convert observations into facts, apply diagnostic knowledge, and produce an explainable fault conclusion and a recommended service action.

Objectives

- Identify important vehicle entities, symptoms, components and diagnostic conclusions.
- Represent the same knowledge using production rules, propositional logic, first-order logic and Prolog.
- Demonstrate forward chaining and backward chaining using clear inference sequences.
- Compare the modelling approaches in expressiveness, inference, complexity, scalability, flexibility and explainability.
- Select the most suitable representation for an automobile diagnostic expert system.



Automobile Fault Diagnosis - Knowledge-Based Modelling Architecture

Figure 1. Overall architecture of the automobile fault diagnosis knowledge model.

2. Domain Knowledge & Facts

The domain is restricted to common observable symptoms and rule-based service decisions. The model is illustrative and does not replace professional mechanical inspection.

2.1 Entities and Attributes

Entity	Attributes / Observations	Examples
Vehicle	vehicle_id, engine_status, mileage_status	V1, overheating
Engine	temperature, starting condition, noise	high temperature
Battery	voltage, charge condition	low voltage

Cooling System	coolant level, fan condition	low coolant
Fuel System	fuel supply, filter condition	blocked filter
Ignition System	spark condition	weak/no spark
Warning Indicator	indicator state	check-engine ON
Diagnosis	fault and recommended action	cooling-system fault

2.2 Initial Facts

- overheating(V1)
- coolant_low(V1)
- fan_not_working(V1)
- starting_failure(V2)
- battery_low(V2)
- no_crank(V2)
- abnormal_noise(V3)
- low_mileage(V3)
- warning_light_on(V3)
- engine_knock(V3)
- air_filter_dirty(V3)
- oil_low(V3)

2.3 Diagnostic Knowledge

Examples: overheating combined with low coolant suggests a cooling-system problem; starting failure with low battery voltage suggests a battery-related fault; abnormal knocking noise with low engine oil suggests an engine lubrication-related problem; low mileage with a dirty air filter suggests restricted air intake; a warning light together with relevant symptoms suggests further diagnostic scanning.

3. Production Rule Model

Production rules are expressed as IF–THEN statements. Each rule has a condition part (antecedent) and an action/conclusion part (consequent).

Rule	Production Rule
R1	IF overheating(V) AND coolant_low(V) THEN cooling_system_fault(V).
R2	IF overheating(V) AND fan_not_working(V) THEN cooling_system_fault(V).

R3	IF starting_failure(V) AND battery_low(V) THEN battery_fault(V).
R4	IF starting_failure(V) AND no_crank(V) THEN starting_system_fault(V).
R5	IF abnormal_noise(V) AND engine_knock(V) AND oil_low(V) THEN lubrication_fault(V).
R6	IF low_mileage(V) AND air_filter_dirty(V) THEN air_intake_fault(V).
R7	IF warning_light_on(V) THEN diagnostic_scan_required(V).
R8	IF cooling_system_fault(V) THEN inspect_coolant_and_radiator(V).
R9	IF battery_fault(V) THEN test_battery_and_charging_system(V).
R10	IF lubrication_fault(V) THEN check_engine_oil_and_internal_wear(V).
R11	IF air_intake_fault(V) THEN clean_or_replace_air_filter(V).
R12	IF diagnostic_scan_required(V) THEN retrieve_obd_fault_codes(V).

Example derivation: From overheating(V1) and coolant_low(V1), R1 fires and derives cooling_system_fault(V1). Then R8 derives inspect_coolant_and_radiator(V1).

4. Propositional Logic Model

For a specific vehicle, V1, each statement is treated as a proposition that is either true or false. This representation is simple but does not naturally express variables or relationships among many vehicles.

Symbol	Meaning
O	Engine is overheating.
C	Coolant level is low.
F	Cooling fan is not working.
S	Vehicle has starting failure.
B	Battery voltage is low.
N	Vehicle has abnormal engine noise.
K	Engine knocking is present.
L	Engine oil level is low.
M	Vehicle has low mileage.
A	Air filter is dirty.
W	Warning light is ON.

D

Cooling-system fault is diagnosed.

Logical Rules

- $O \wedge C \rightarrow D$
- $O \wedge F \rightarrow D$
- $S \wedge B \rightarrow \text{BatteryFault}$
- $N \wedge K \wedge L \rightarrow \text{LubricationFault}$
- $M \wedge A \rightarrow \text{AirIntakeFault}$
- $W \rightarrow \text{ScanRequired}$
- $D \rightarrow \text{InspectCoolingSystem}$

Example Inference

Given $O = \text{True}$ and $C = \text{True}$, $O \wedge C$ is true. By modus ponens, $D = \text{True}$. Therefore, the model concludes that a cooling-system fault is present and inspection is required.

5. First-Order Logic Model

First-Order Logic (FOL) extends propositional logic by allowing variables, predicates, quantifiers and relationships. This is more suitable when the service center diagnoses many vehicles.

Predicates

- $\text{Overheating}(v)$ – vehicle v has an overheating engine.
- $\text{CoolantLow}(v)$ – vehicle v has low coolant.
- $\text{FanFault}(v)$ – vehicle v has a cooling-fan fault.
- $\text{StartingFailure}(v)$ – vehicle v fails to start.
- $\text{BatteryLow}(v)$ – vehicle v has low battery voltage.
- $\text{AbnormalNoise}(v)$ – vehicle v has abnormal noise.
- $\text{EngineKnock}(v)$ – vehicle v has engine knocking.
- $\text{OilLow}(v)$ – vehicle v has low engine oil.
- $\text{LowMileage}(v)$ – vehicle v has low mileage.
- $\text{AirFilterDirty}(v)$ – vehicle v has a dirty air filter.
- $\text{CoolingFault}(v)$ – vehicle v has a cooling-system fault.
- $\text{Inspect}(v, x)$ – vehicle v requires inspection of component/system x .

FOL Rules

- $\forall v [(\text{Overheating}(v) \wedge \text{CoolantLow}(v)) \rightarrow \text{CoolingFault}(v)]$
- $\forall v [(\text{Overheating}(v) \wedge \text{FanFault}(v)) \rightarrow \text{CoolingFault}(v)]$
- $\forall v [(\text{StartingFailure}(v) \wedge \text{BatteryLow}(v)) \rightarrow \text{BatteryFault}(v)]$
- $\forall v [(\text{AbnormalNoise}(v) \wedge \text{EngineKnock}(v) \wedge \text{OilLow}(v)) \rightarrow \text{LubricationFault}(v)]$
- $\forall v [(\text{LowMileage}(v) \wedge \text{AirFilterDirty}(v)) \rightarrow \text{AirIntakeFault}(v)]$
- $\forall v [\text{WarningLightOn}(v) \rightarrow \text{ScanRequired}(v)]$
- $\forall v [\text{CoolingFault}(v) \rightarrow \text{Inspect}(v, \text{cooling_system})]$

Example

Facts: $\text{Overheating}(V1)$ and $\text{CoolantLow}(V1)$. Rule: $\forall v [(\text{Overheating}(v) \wedge \text{CoolantLow}(v)) \rightarrow \text{CoolingFault}(v)]$. Substituting $v = V1$ gives $\text{CoolingFault}(V1)$. Hence the system can reason about any vehicle without writing a separate rule for each vehicle.

6. Prolog Model & Implementation

The Prolog model uses facts for observations and rules for diagnostic relationships. The following program is directly suitable for a standard Prolog environment such as SWI-Prolog.

```
% Automobile Fault Diagnosis
% Facts
overheating(v1).
coolant_low(v1).
fan_not_working(v1).

starting_failure(v2).
battery_low(v2).
no_crank(v2).

abnormal_noise(v3).
engine_knock(v3).
oil_low(v3).
low_mileage(v3).
air_filter_dirty(v3).
warning_light_on(v3).

% Diagnostic rules
fault(V, cooling_system_fault) :-
    overheating(V), coolant_low(V).
fault(V, cooling_system_fault) :-
    overheating(V), fan_not_working(V).

fault(V, battery_fault) :-
    starting_failure(V), battery_low(V).

fault(V, starting_system_fault) :-
    starting_failure(V), no_crank(V).

fault(V, lubrication_fault) :-
    abnormal_noise(V), engine_knock(V), oil_low(V).

fault(V, air_intake_fault) :-
    low_mileage(V), air_filter_dirty(V).

fault(V, diagnostic_scan_required) :-
    warning_light_on(V).

% Service recommendations
recommend(V, inspect_cooling_system) :-
    fault(V, cooling_system_fault).

recommend(V, test_battery_and_charging_system) :-
    fault(V, battery_fault).

recommend(V, check_engine_oil_and_internal_wear) :-
    fault(V, lubrication_fault).

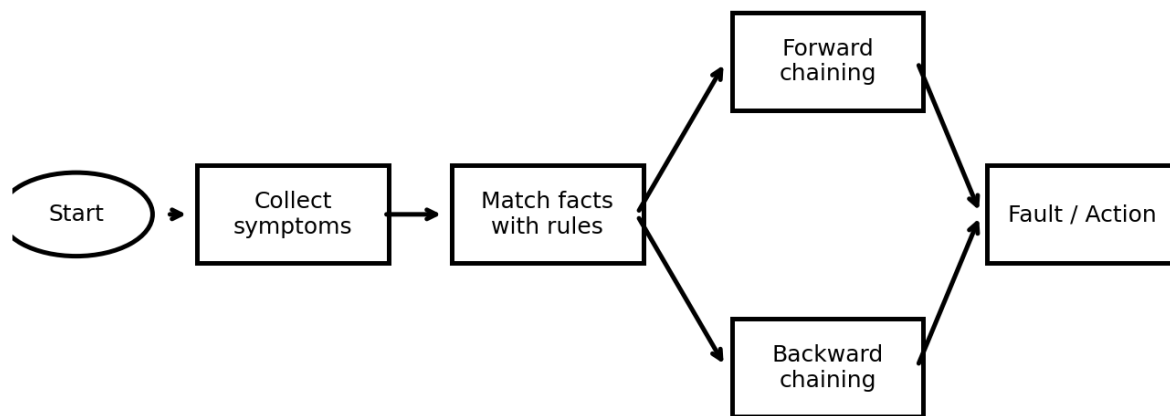
recommend(V, clean_or_replace_air_filter) :-
    fault(V, air_intake_fault).

recommend(V, retrieve_obd_fault_codes) :-
    fault(V, diagnostic_scan_required).
```

Sample Queries and Expected Outputs

Query	Expected Output
?- fault(v1, F).	F = cooling_system_fault.
?- recommend(v1, Action).	Action = inspect_cooling_system.
?- fault(v2, F).	F = battery_fault ; F = starting_system_fault.
?- fault(v3, F).	F = lubrication_fault ; F = air_intake_fault ; F = diagnostic_scan_required.
?- recommend(v3, Action).	Action = check_engine_oil_and_internal_wear ; Action = clean_or_replace_air_filter ; Action = retrieve_obd_fault_codes.

7. Forward & Backward Chaining Demonstration



Forward and Backward Chaining - Reasoning Flow

Figure 2. Reasoning flow for forward and backward chaining.

7.1 Forward Chaining

Forward chaining is data-driven: start with known facts and repeatedly fire applicable rules.

1. Initial facts for V1: overheating(V1), coolant_low(V1), fan_not_working(V1).
2. R1 matches overheating(V1) + coolant_low(V1) → cooling_system_fault(V1).
3. R8 matches cooling_system_fault(V1) → inspect_coolant_and_radiator(V1).
4. Conclusion: V1 has a cooling-system fault; cooling-system inspection is recommended.

7.2 Backward Chaining

Backward chaining is goal-driven: start with a desired conclusion and work backward to find facts that satisfy its rule conditions.

5. Goal: cooling_system_fault(V1).

6. Select R1: cooling_system_fault(V1) requires overheating(V1) and coolant_low(V1).
7. Check the fact base: both facts are available.
8. Therefore R1 succeeds and the goal cooling_system_fault(V1) is proven.
9. Then R8 can be used to derive inspect_coolant_and_radiator(V1).

Forward chaining is useful when many observations arrive and several conclusions may be discovered. Backward chaining is efficient when the service center has a specific suspected fault or question to verify.

8. Comparative Analysis of Modelling Approaches

Parameter	Production Rules	Propositional Logic	First-Order Logic	Prolog
Knowledge Representation	IF–THEN rules and facts	Boolean propositions	Predicates, variables, relations	Facts + rules + executable predicates
Expressiveness	High for procedural diagnosis	Low–Medium	Very high	Very high
Rule Representation	Direct and readable	Implications	General quantified rules	Horn clauses
Inference Mechanism	Usually forward/backward engine	Logical entailment	Unification + logical inference	Backward chaining with backtracking
Forward Chaining	Natural and common	Possible but external	Possible	Not native; can be simulated
Backward Chaining	Possible	Possible but less direct	Possible	Native and efficient
Handling Relationships	Limited unless extended	Weak	Strong	Strong
Scalability	Good for moderate rule bases	Poor for large structured domains	Good with structured knowledge	Good for rule-based diagnostic systems
Explainability	Excellent	Good for simple cases	Good	Excellent due to rules and proof search
Ease of Implementation	Easy	Easy	Moderate	Moderate
Suitability for Problem	High	Moderate	Very high	Very high

8.1 Summary of Strengths and Limitations

- Production Rules: highly readable and close to how service technicians express diagnostic knowledge, but relationships and variables are less natural.
- Propositional Logic: mathematically simple and suitable for small fixed cases, but the number of propositions grows rapidly when many vehicles and components are introduced.
- First-Order Logic: provides variables, quantifiers and relationships, making it the most expressive formal representation in this comparison; implementation requires a suitable inference mechanism.
- Prolog: combines logical representation with executable inference, unification and backtracking. It is especially suitable for a working rule-based prototype and query-driven diagnosis.

9. Results, Discussion & Recommended Model

9.1 Results

Vehicle	Derived Faults	Recommended Actions
V1	Cooling-system fault	Inspect coolant, radiator and cooling fan

V2	Battery fault; starting-system fault	Test battery/charging system and starting circuit
V3	Lubrication fault; air-intake fault; scan required	Check oil/internal wear, service air filter, retrieve OBD codes

9.2 Discussion

The four approaches can represent the same core diagnostic knowledge, but they differ in generality and execution. Propositional logic is adequate for a small fixed demonstration. Production rules are highly understandable for service-center knowledge. FOL gives the strongest formal representation of multiple vehicles, components and relationships. Prolog provides the best combination of representation and executable reasoning for a compact expert-system implementation.

9.3 Recommended Model

Prolog is recommended as the primary implementation model, supported by production rules and FOL during knowledge design. Prolog is suitable because the domain naturally consists of facts, Horn-clause rules, queries and diagnostic conclusions. Backward chaining allows the system to answer a specific diagnostic question efficiently, while the rule base remains readable and explainable.

Limitations and Improvements

- The model uses simplified symptoms and deterministic rules; real vehicle diagnosis may involve sensor uncertainty and multiple interacting faults.
- Future versions can add confidence scores or fuzzy reasoning for symptoms such as 'slightly high temperature' or 'unusual noise'.
- An OBD-II interface could automatically collect diagnostic codes and sensor readings.
- A larger knowledge base can cover transmission, brakes, electrical systems, fuel injection and emissions.
- Conflict-resolution strategies can be added when multiple rules produce competing diagnoses.

10. Conclusion, References & GitHub

This assessment modelled automobile fault diagnosis using production rules, propositional logic, first-order logic and Prolog. Forward chaining was demonstrated as data-driven reasoning, while backward chaining was demonstrated as goal-driven reasoning. The comparison shows that propositional logic is simplest, production rules are highly readable, FOL is the most expressive formal model for relationships and variables, and Prolog is the most practical executable choice for this rule-based expert-system prototype.

Suggested GitHub Repository Structure

```

automobile-fault-diagnosis/
├── README.md
├── prolog/
│   └── automobile_diagnosis.pl
├── models/
│   ├── production_rules.txt
│   ├── propositional_logic.txt
│   └── first_order_logic.txt
├── test_cases/
│   └── sample_queries.txt
└── diagrams/
    ├── architecture.png
    └── chaining_flow.png

```

GitHub Repository Link: <https://github.com/192525185simats-cyber/SLOT-D-MLA0104-AI/blob/main/assessment/CO5/co5%20at3>

References

10. Stuart Russell and Peter Norvig, Artificial Intelligence: A Modern Approach, Pearson.
11. Ian Sommerville, Software Engineering, Pearson.
12. SWI-Prolog Documentation, Prolog language reference and examples.
13. Course notes: Artificial Intelligence and Expert Systems, CO5 – Rule-based expert systems and inference.

Evaluation Rubrics – Self Assessment

Criterion	Weightage	Self-Evaluation	Marks Awarded	Justification
1. Problem Analysis and Domain Understanding	10%	Excellent	10/10	Clear problem, domain entities, constraints and outcomes are defined.
2. Domain Knowledge and Fact Identification	10%	Excellent	10/10	Facts, symptoms, components, relationships and conditions are identified.
3. Production Rule Modelling	15%	Excellent	15/15	12 consistent IF–THEN rules cover diagnosis and recommendations.
4. Propositional Logic Modelling	10%	Excellent	10/10	Propositions, connectives and inference examples are included.
5. First-Order Logic Modelling	15%	Excellent	15/15	Predicates, variables, quantification and generalized rules are provided.
6. Prolog Modelling and Implementation	15%	Excellent	15/15	Facts, rules, predicates, queries and expected outputs are included.
7. Forward and Backward Chaining Demonstration	10%	Excellent	10/10	Both mechanisms are shown step-by-step with inference sequences.
8. Comparative Analysis	10%	Excellent	10/10	Required parameters are compared across all four approaches.
9. Results, Discussion and Model Selection	3%	Excellent	3/3	Results are summarized and Prolog is justified as the recommended model.
10. Documentation, GitHub Repository and Presentation	2%	Good*	1.5/2	Report is structured; GitHub link must be added after repository upload.

Estimated Total: 99.5 / 100

*The final 0.5 mark depends on creating and attaching the actual GitHub repository link and completing the required presentation.

Student Signature: S. Roshni